

attacker can gain complete access over the kernel of the victim machine. In order to prevent buffer overflows, one needs to know more about them, particularly how and why they occur. Let us use GDB to uncover the possible buffer overflows in program code.

GDB: a quick tutorial

GDB is an open source debugger that can be used on most UNIX systems and also on some variants of Windows. Apart from disassembling code and displaying the assembly, it also helps programmers analyse the stack and memory. Launch it from the terminal with *gdb* and it will display information about GDB, specifically its version:

```
GNU gdb (Ubuntu/Linaro 7.4-2012.04-0ubuntu2.1) 7.4-2012.04
Copyright (C) 2012 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type
"show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
For bug reporting instructions, please see:
<http://bugs.launchpad.net/gdb-linaro/>.
(gdb)
```

Basic commands

Let us first go over some of the basic commands that will be used in this article. The full form of the command and its parameter(s) are followed by the short form accepted by GDB, in parentheses.

disassemble <function name> (disas)

This command displays the disassembly of the specified function—code obtained by converting object code to assembly language (human readable) code. For example, consider the following simple piece of code:

```
//hello_world.c
#include <stdio.h>

int main (int argc, char ** argv) {
    printf ("Hello World \n");
    return 0;
}
```

Now compile this code and start GDB:

```
$ gcc -gdb hello_world.c -o hello_world
$ gdb hello_world
```

The disassembly for the main function looks like what's shown below:

```
(gdb) disassemble main
Dump of assembler code for function main:
0x000483d4 <+0>: push %ebp
0x000483d5 <+1>: mov %esp,%ebp
0x000483d7 <+3>: and $0xfffff0,%esp
0x000483da <+6>: sub $0x10,%esp
0x000483dd <+9>: movl $0x0484c0, (%esp)
0x000483e4 <+16>: call 0x00482f0 <puts@plt>
0x000483e9 <+21>: mov $0x6,%eax
0x000483ee <+26>: leave
0x000483ef <+27>: ret
End of assembler dump.
(gdb)
```

The first column shows the memory address, the second shows the offset of the memory of the corresponding instruction relative to the start of the program, and the third column shows the instruction.

break <position> (or b)

This helps set breakpoints in the program—the pauses while debugging. The position could be a line number or a memory address. In the following examples, the first sets a breakpoint at the fifth line of code, and the second at a specific memory address (0x000483e4):

```
(gdb) break 5
(gdb) break *0x000483e4
```

step (or s)

This command steps through each instruction in the code. Another similar instruction is *next* (or *n*). The difference between the two is that *next* steps over functions, but *step* steps into them.

examine (x)

This very important command helps us to view the contents of a particular memory location, in hexadecimal, decimal, binary or ASCII formats. There's also an option to view the data as bytes, half-bytes or words, etc. In this article, we inspect memory contents in the hexadecimal format in words.

run <command line argument>

This command executes the program from within GDB.

info registers <register> (or i r)

This command helps us view the contents of the specified register. A register is a variable of 32 bits where values are stored in the memory. These are used to store values like the